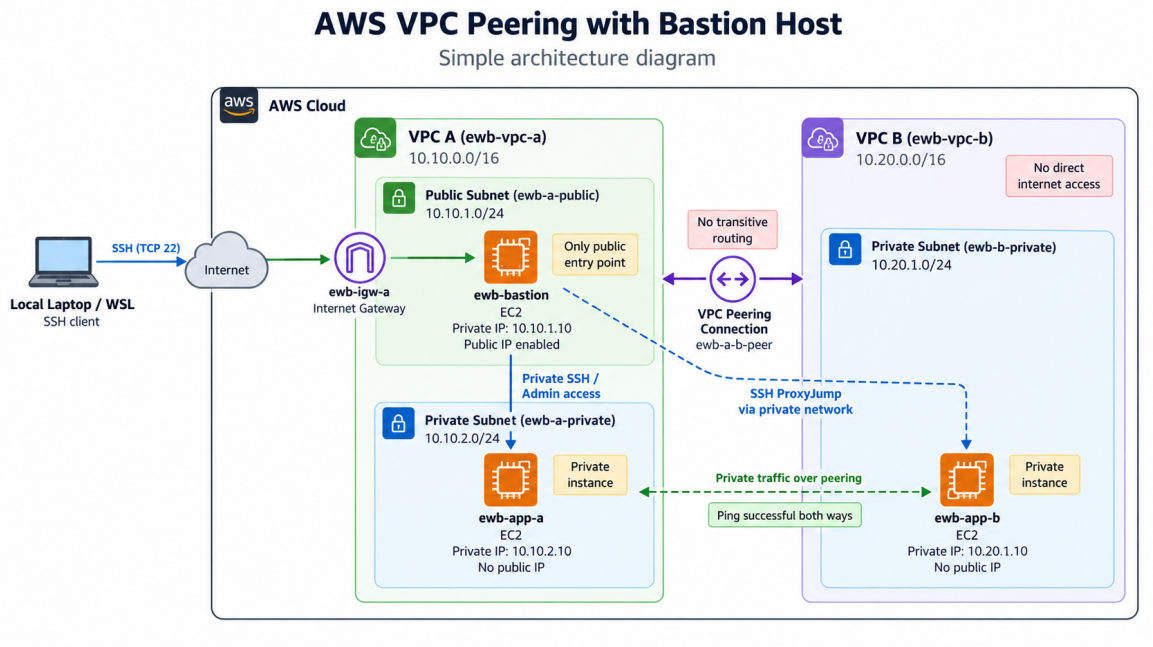
AWS VPC Peering with Bastion Host

Secure Private Communication Across Two Amazon VPCs



Student Name	Ketan Vijay Dhanvijay
Roll Number	MC250124CA
College	JD College of Engineering & Management
AWS Region	Asia Pacific (Sydney) - ap-southeast-2
Project Type	Cloud Computing / AWS Networking Project
GitHub	github.com/Ketankd519/aws-vpc-peering-bastion-lab
Submission Date	30 October 2026

1. Abstract

This project implements secure private communication between two independent Amazon Virtual Private Clouds (VPCs) using VPC Peering. VPC A contains a public bastion host and a private application server, while VPC B contains a private application server only. Administrative SSH access is performed through the bastion host using SSH ProxyJump, and application-to-application communication uses private IPv4 addresses across the peering connection. The implementation demonstrates VPC design, subnetting, Internet Gateway use, custom route tables, least-privilege security groups, bidirectional routing, validation testing, controlled failure/recovery, and final AWS resource cleanup.

Project outcome: App-A (10.10.2.10) and App-B (10.20.1.10) communicated successfully in both directions over VPC Peering, while both application instances remained private.

Objectives

- Create two non-overlapping VPCs and purpose-built subnets.
- Use a public bastion as the only administrative entry point.
- Keep App-A and App-B without public IPv4 addresses.
- Configure an active VPC peering connection and explicit forward/return routes.
- Restrict SSH and ICMP traffic using host-specific /32 security-group rules.
- Prove connectivity before and after peering, including controlled failure and recovery.
- Clean up the AWS lab resources after collecting evidence.

Technology Used

Service / Tool	Purpose
Amazon VPC	Isolated virtual networks
Amazon EC2	Bastion, App-A and App-B instances
VPC Peering	Private VPC-to-VPC connectivity
Route Tables	Forward and return traffic paths
Security Groups	Stateful firewall rules
Internet Gateway	Internet access for bastion only
OpenSSH / ProxyJump	Administration of private instances
WSL Ubuntu	Local SSH client environment

2. Architecture and Network Design

The design uses two VPCs with non-overlapping CIDRs. The bastion host is placed in VPC A public subnet. App-A and App-B are private EC2 instances. VPC Peering carries private traffic directly between the VPCs; it does not provide transitive routing or internet access to VPC B.

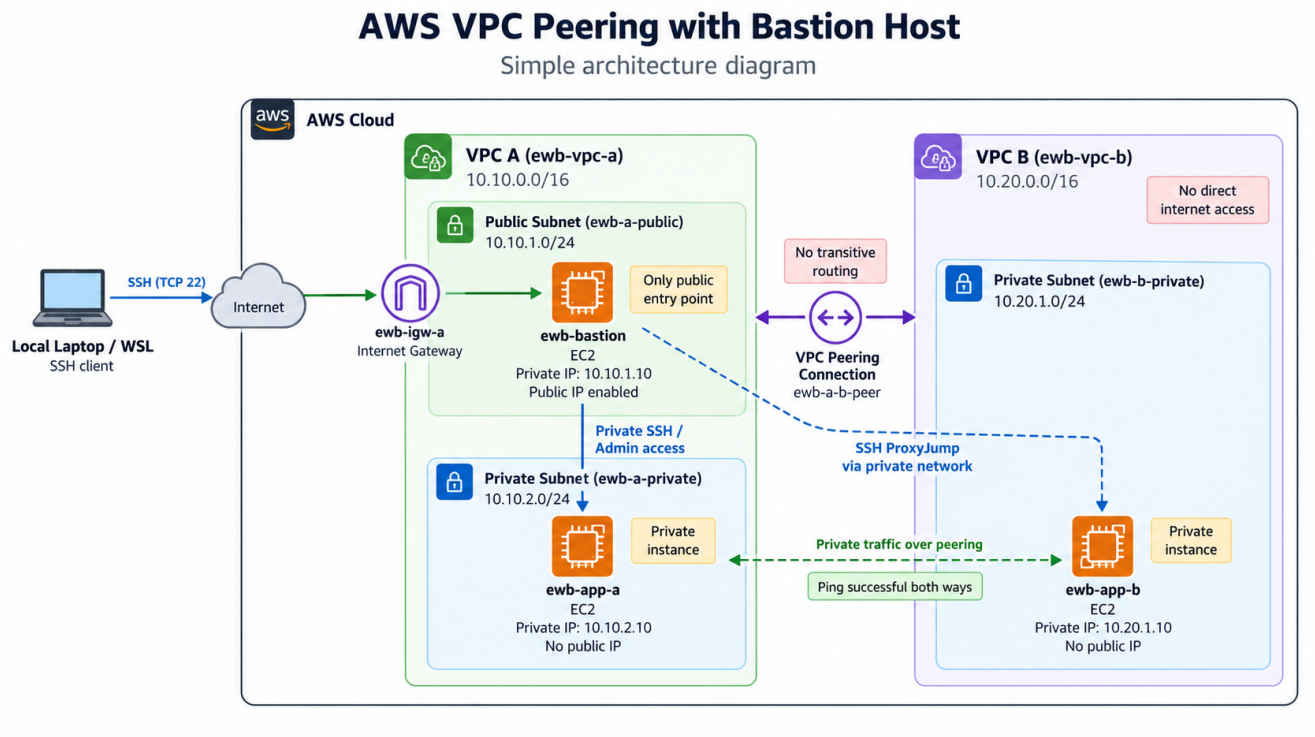


Figure 1. Final AWS VPC peering architecture.

Resource Inventory

Resource	Name	CIDR / IP	ID
VPC A	ewb-vpc-a	10.10.0.0/16	vpc-02364be552b241cc3
Public Subnet A	ewb-a-public	10.10.1.0/24	subnet-0e538871c92823fac
Private Subnet A	ewb-a-private	10.10.2.0/24	subnet-0379bbaa2849fbce9
VPC B	ewb-vpc-b	10.20.0.0/16	vpc-00b24dcb6814622c6
Private Subnet B	ewb-b-private	10.20.1.0/24	subnet-06fbb9e462d365069
Internet Gateway	ewb-igw-a	-	igw-0b518bcc91e3dcb59
VPC Peering	ewb-a-b-peer	A <-> B	pcx-033fa7491110d439d

3. EC2 and Subnet Configuration

Instance	Instance ID	Private IP	Placement / Public IP
ewb-bastion	i-036d813be0acc1667	10.10.1.10	VPC A public subnet / public IPv4
ewb-app-a	i-0b55f898de46d52eb	10.10.2.10	VPC A private subnet / none
ewb-app-b	i-0d56614df7b7d44d6	10.20.1.10	VPC B private subnet / none

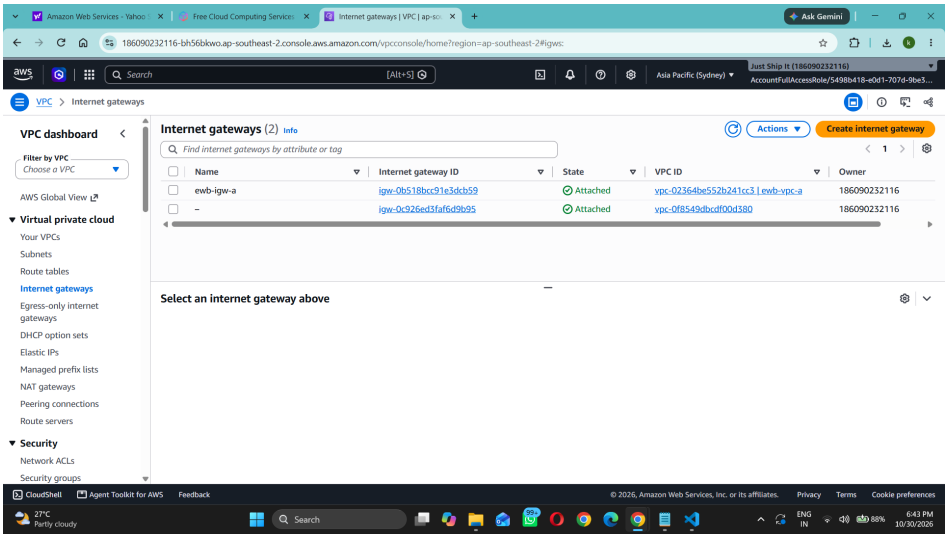


Figure 2. Subnet evidence showing the project subnets and CIDR assignments.

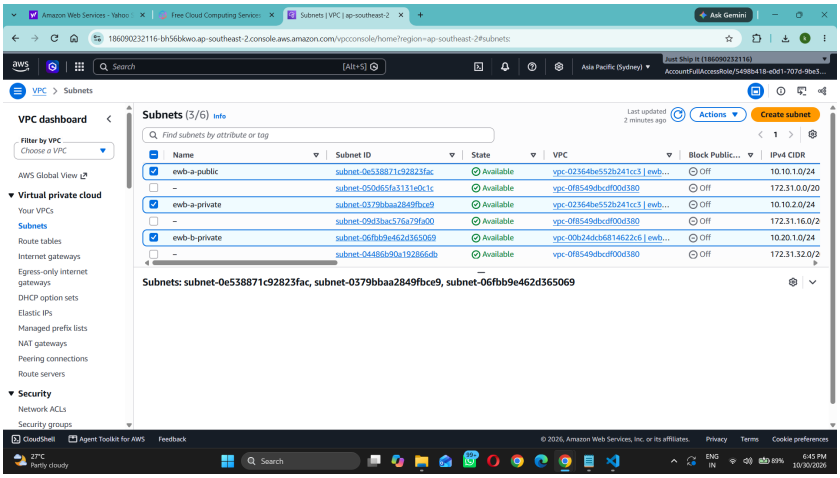


Figure 3. Internet Gateway ewb-igw-a attached to VPC A during the implementation.

4. Routing and VPC Peering

The peering connection became Active only after the request was explicitly accepted. Connectivity then required three peering routes: VPC A private to VPC B, VPC B private to VPC A, and VPC A public to VPC B so that the bastion could reach App-B through ProxyJump.

Route Table	Associated Subnet	Routes
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 -> local; 0.0.0.0/0 -> ewb-igw-a; 10.20.0.0/16 -> peering
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 -> local; 10.20.0.0/16 -> peering
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 -> local; 10.10.0.0/16 -> peering

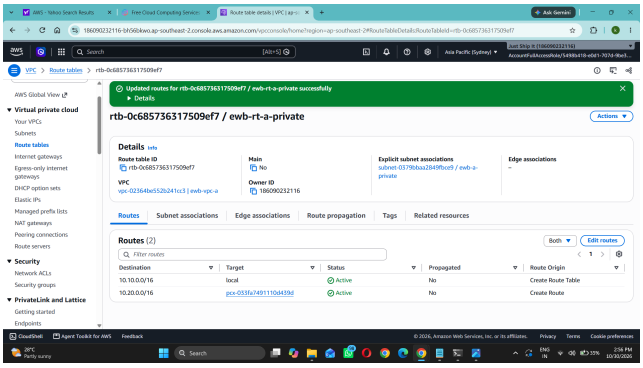


Figure 4. VPC A private route to VPC B.

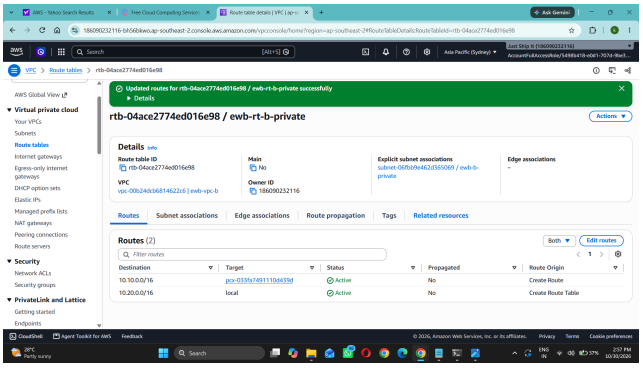


Figure 5. VPC B private return route to VPC A.

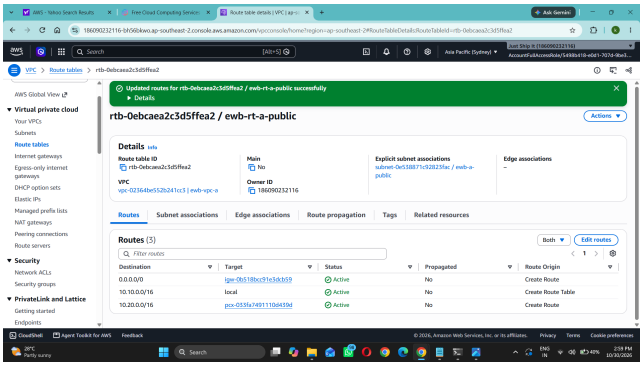


Figure 6. VPC A public route table including the peering route.

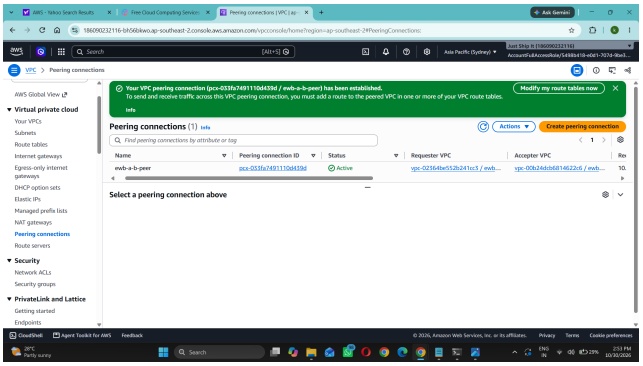


Figure 7. VPC peering connection ewb-a-b-peer in Active state.

5. Security Group Design

Security groups were restricted to the exact hosts required for this lab. No application instance was opened to the public internet.

Security Group	Inbound Rule	Source
ewb-sg-bastion	SSH TCP 22	Administrator current public IPv4 /32
ewb-sg-app-a	SSH TCP 22	10.10.1.10/32
ewb-sg-app-a	ICMP Echo Request	10.20.1.10/32
ewb-sg-app-b	SSH TCP 22	10.10.1.10/32
ewb-sg-app-b	ICMP Echo Request	10.10.2.10/32

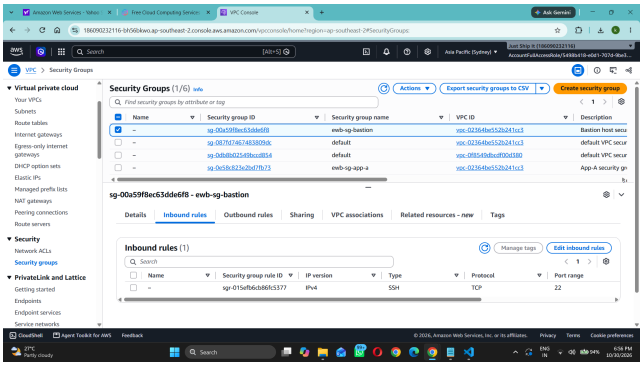


Figure 8. Bastion inbound rule: SSH 22 from administrator public IP /32.

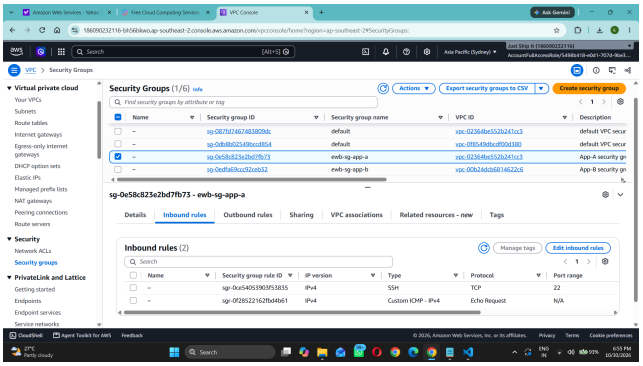


Figure 9. App-A inbound rules: SSH from bastion and ICMP from App-B.

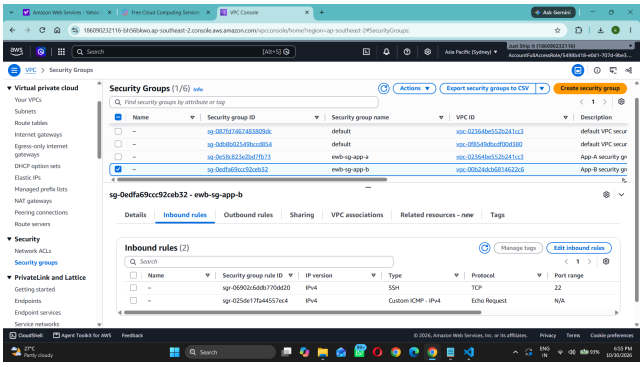


Figure 10. App-B inbound rules: SSH from bastion and ICMP from App-A.

6. Secure Administration with Bastion and ProxyJump

The private key remained only on the local WSL machine. It was not copied to the bastion or application instances. SSH ProxyJump was used so the local SSH client authenticated to the private instances through the bastion.

SSH configuration used:

```
Host ewb-bastion
HostName <BASTION_PUBLIC_IP>
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem

Host ewb-app-a
HostName 10.10.2.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
ProxyJump ewb-bastion

Host ewb-app-b
HostName 10.20.1.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
ProxyJump ewb-bastion
```

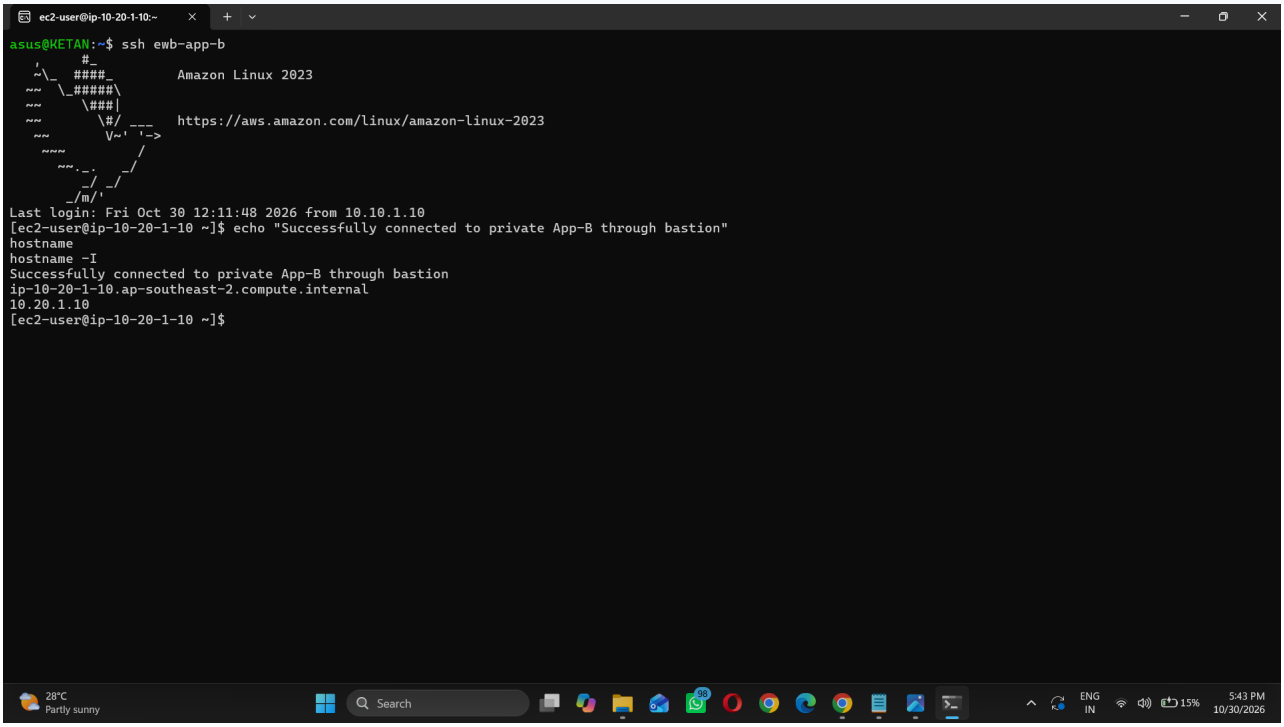


Figure 11. Successful SSH connection to private App-B (10.20.1.10) through the bastion host.

7. Connectivity Validation

Test	Source	Destination	Expected / Result
Baseline before peering	App-A 10.10.2.10	App-B 10.20.1.10	FAIL - 100% packet loss
Forward after peering	App-A 10.10.2.10	App-B 10.20.1.10	PASS - 0% packet loss
Reverse after peering	App-B 10.20.1.10	App-A 10.10.2.10	PASS - 0% packet loss
Private SSH	Local WSL via bastion	App-B 10.20.1.10	PASS

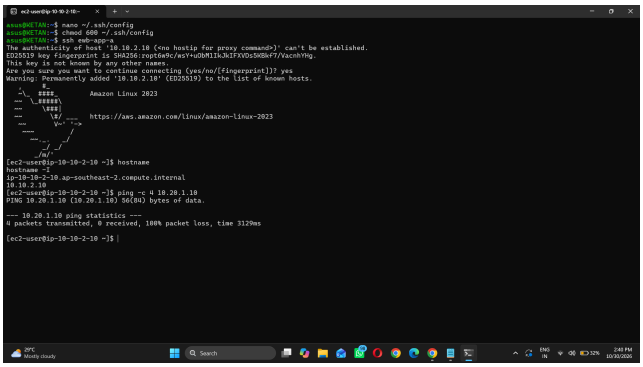


Figure 12. Baseline: App-A to App-B fails before peering/routes are configured.

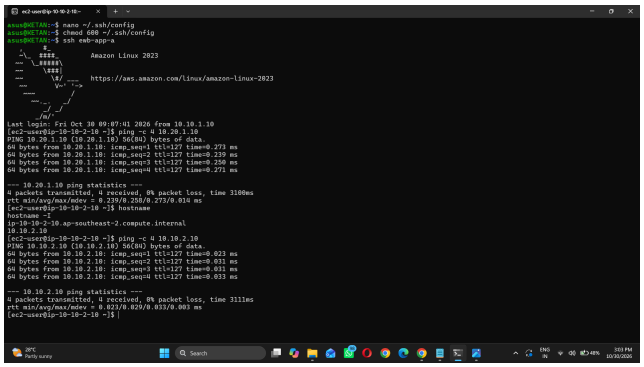


Figure 13. Forward path: App-A reaches App-B after peering.

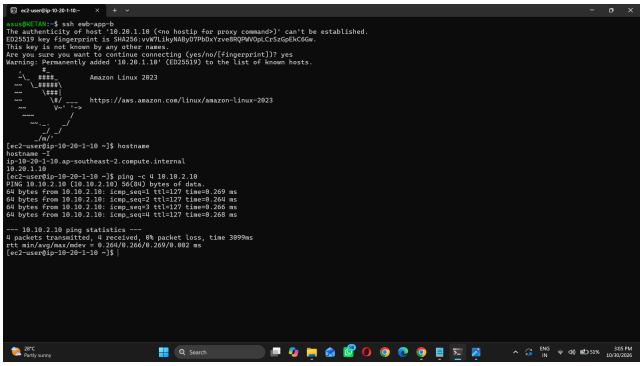


Figure 14. Reverse path: App-B reaches App-A successfully.

8. Controlled Failure and Recovery

To demonstrate that an Active peering connection is not sufficient by itself, the App-B ICMP Echo Request rule from 10.10.2.10/32 was temporarily removed. The same App-A to App-B ping then failed. After restoring the rule, the exact test succeeded again.

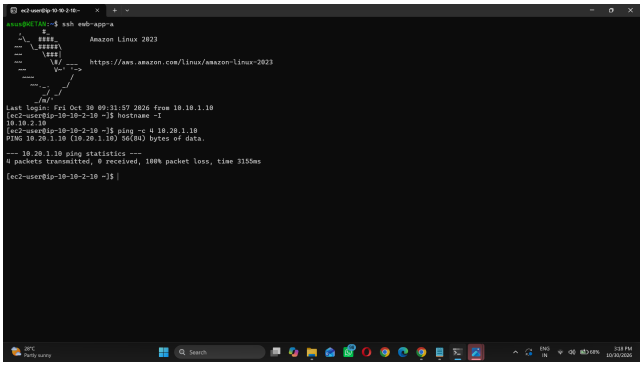


Figure 15. Controlled failure: 100% packet loss after removing App-B ICMP permission.

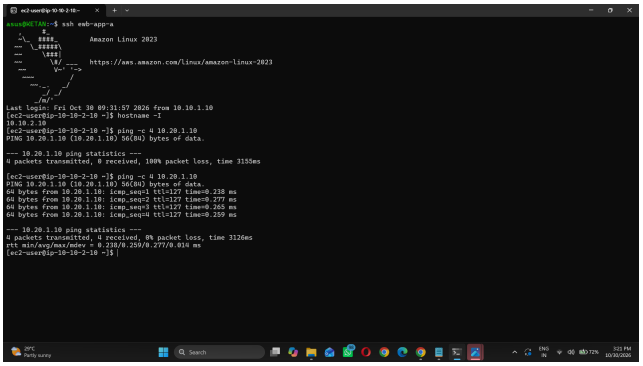


Figure 16. Recovery: 0% packet loss after restoring the ICMP rule.

Conclusion from the test: Successful VPC-to-VPC communication depends on all three layers being correct: an Active peering connection, matching route-table entries in both directions, and security-group rules permitting the traffic.

9. Troubleshooting Performed

During bastion SSH testing, the connection initially timed out on TCP port 22. Network routing and the Internet Gateway were verified. The root cause was the bastion security group allowing an older administrator public IPv4 address. After updating the SSH source to the current public IPv4 /32, TCP port 22 became reachable and SSH succeeded.

Diagnostic commands used:

```
curl -4 ifconfig.me
nc -vz <BASTION_PUBLIC_IP> 22
ssh -vvv -i ~/.ssh/ewb-lab.pem ec2-user@<BASTION_PUBLIC_IP>
```

This troubleshooting demonstrated the difference between SSH authentication problems and basic network reachability problems: a TCP timeout occurs before the SSH private key is evaluated.

Implementation Notes

• Amazon Linux 2023 was used for all three project instances.

• Private IPs were intentionally fixed for clear, reproducible security-group rules and evidence.

• VPC B had no Internet Gateway and no public-facing instance.

• The public route table in VPC A included a peering route because the bastion itself needed a path to App-B for ProxyJump.

10. AWS Resource Cleanup and Evidence

After all tests and screenshots were completed, the lab resources were cleaned up to avoid unnecessary AWS charges. Cleanup evidence is also stored in the GitHub repository.

Cleanup Item	Evidence Status
EC2: ewb-bastion, ewb-app-a, ewb-app-b	Terminated
Peering routes	Removed
VPC peering connection ewb-a-b-peer	Deleted
Project security groups	Deleted
Project subnets	Deleted
Custom route tables	Deleted
Internet Gateway ewb-igw-a	Detached during cleanup (evidence captured)
Key pair ewb-lab	Deleted
Project VPCs	Deleted / project VPC cleanup evidenced

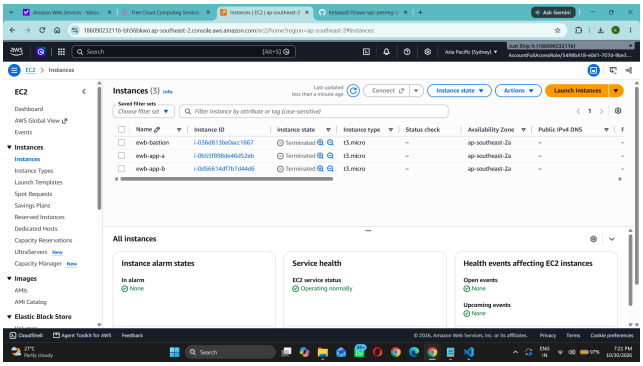


Figure 17. All three project EC2 instances terminated.

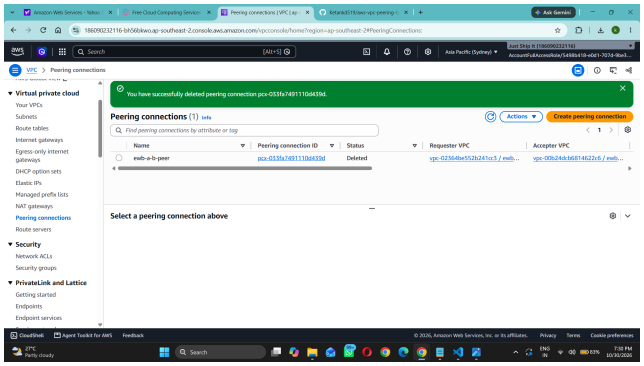


Figure 18. VPC peering connection deleted.

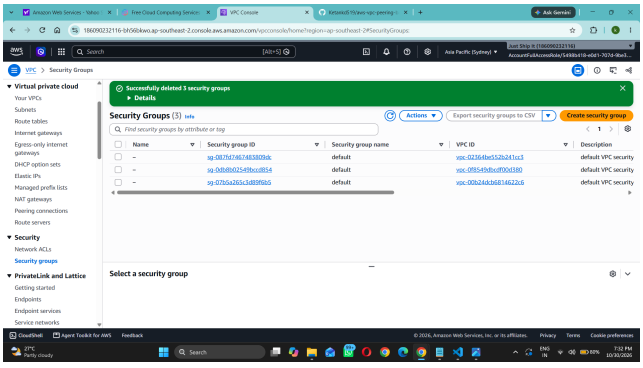


Figure 19. Project security groups removed; only default groups remain.

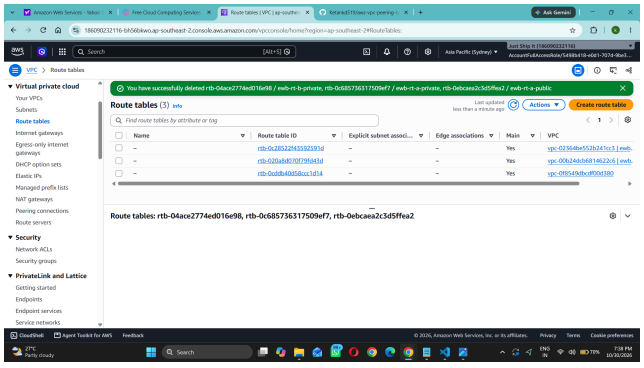


Figure 20. Custom project route tables deleted.

11. Results, Learning Outcomes and Conclusion

Final result: The AWS VPC Peering lab was successfully implemented and validated. App-A and App-B communicated bidirectionally using private IPv4 addresses, and private SSH administration to App-B succeeded through the public bastion host without assigning public IPs to the application servers.

Key Learning Outcomes

• VPC peering requires non-overlapping CIDR blocks.

• Creating a peering connection does not automatically create route-table entries.

• Both forward and return routes are required for successful communication.

• Security groups continue to enforce traffic policy across an Active peering connection.

• A bastion host provides a controlled entry point for private workloads.

• SSH ProxyJump avoids copying private keys to intermediate servers.

• VPC peering is non-transitive and does not allow VPC B to use VPC A Internet Gateway for internet access.

• Controlled failure testing is useful for proving that the intended security control is actually responsible for traffic behavior.

Final Status

Area	Status
Architecture and subnetting	Completed
EC2 and bastion deployment	Completed
VPC Peering	Completed
Forward and reverse routing	Passed
Private SSH to App-B	Passed
Controlled failure / recovery	Passed
Security-rule evidence	Completed
AWS cleanup	Completed with evidence
GitHub documentation	Completed

GitHub Repository: github.com/Ketankd519/aws-vpc-peering-bastion-lab

Project Result: Successfully Completed

References

1. Amazon Web Services documentation - Amazon VPC, VPC Peering, Route Tables, Security Groups and EC2.
2. EWB Courses - AWS VPC Peering Student Project workbook supplied for the lab.